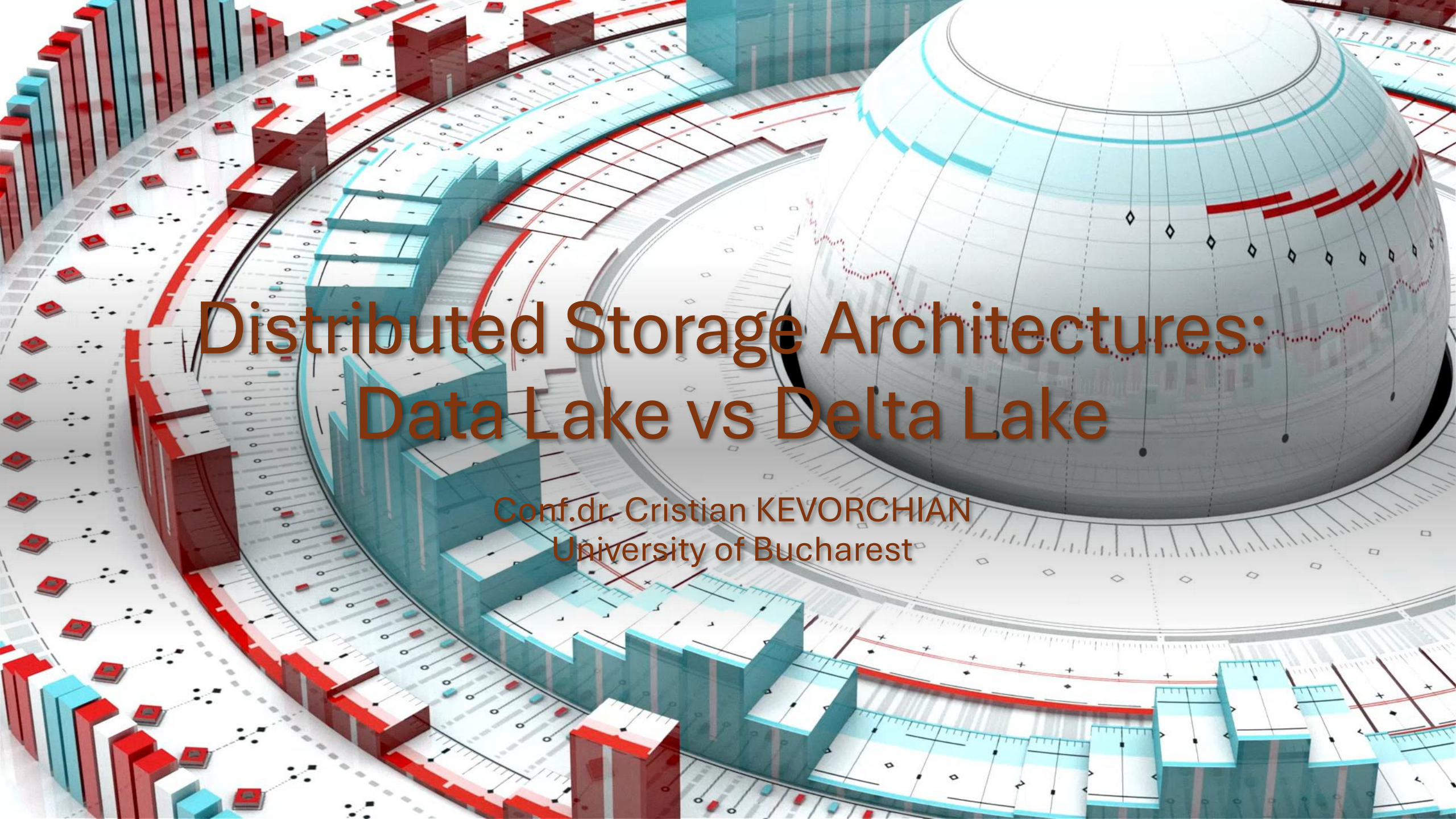


The background is a complex 3D visualization. It features a large, semi-transparent dome on the right side, which has a grid pattern and a red line graph with diamond markers wrapped around its surface. To the left of the dome are several concentric rings of data blocks. These blocks are represented as 3D rectangular prisms in red and light blue. They are arranged in a way that suggests a hierarchical or layered structure, with some blocks appearing to be stacked or connected by lines. The overall scene is set against a light gray background with a grid of small black dots and lines, giving it a technical, data-centric feel.

Distributed Storage Architectures: Data Lake vs Delta Lake

Conf.dr. Cristian KEVORCHIAN
University of Bucharest

An abstract digital cityscape rendered in shades of blue. The scene features several 3D rectangular blocks of varying sizes, some of which are hollow or have glowing interiors. The surfaces of these blocks are covered in a pattern of binary code (0s and 1s). Bright blue and red light beams emanate from various points on the blocks, creating a sense of dynamic energy and data flow. The background is a deep blue gradient, and the overall composition suggests a high-tech, data-driven environment.

Definition and Role of Distributed Storage in Cloud-Native Data Systems

Distributed storage is a **data architecture paradigm** where data is stored across multiple physical or virtual nodes, often spanning different servers, data centers, or geographic regions.

Unlike traditional centralized storage systems, distributed storage systems are designed to *scale horizontally* and operate reliably in cloud-native environments.

CAP Theorem Classification of Azure Distributed Storage Services

CAP Theorem	Azure Service	Consistency	Availability	Partition Tolerance	Typical Use Case
PC	Azure SQL Database	Strong (ACID)	May degrade during failover	Regional replication	Transactional systems, relational data integrity
PC	Cosmos DB (Strong mode)	Strong	May sacrifice availability	Global distribution	Globally distributed apps needing strict consistency
PA	Blob Storage / Data Lake Gen2	Strong (read/write)	High (ZRS, GRS)	Built-in	Big data, ML pipelines, file-based analytics
PA	Cosmos DB (Eventual mode)	Eventual	SLA-backed	Global distribution	Real-time apps, social feeds, IoT telemetry
PA	Azure Table Storage	Eventual	High	Built-in	NoSQL key-value storage, scalable metadata
PA	Azure Queue Storage	Eventual	High	Built-in	Messaging systems, decoupled microservices

The Three Pillars of Decentralized Storage in Azure Storage

Pillar	Core Capabilities	Azure Implementation	Role in Decentralization
Data Lake	Stores raw(csv, json sau paquet) unstructured/semi-structured data at scale	Azure Data Lake Storage Gen2	Foundation layer; enables partitioning and horizontal scalability
Delta Lake	Adds ACID transactions, schema enforcement, time travel, and versioning	Delta Lake on Azure Databricks	Ensures consistency and reliability over partitioned data
Lakehouse	Combines Data Lake flexibility with Data Warehouse structure and performance	Unity Catalog + Delta Lake + Azure Synapse/Fabric	Unified analytics; supports structured queries and ML over decentralized data

Main Characteristics

- **Decentralization:** in distributed storage systems, specifically showing how data is partitioned and/or replicated across multiple nodes.
- **Redundancy:** Multiple copies of data ensure fault tolerance.
 - Redundancy refers to the **duplication of critical components or data** within a system to ensure availability and reliability in case of failure. In distributed storage, this typically means storing **multiple copies of data** across different nodes, zones, or regions.
- **Scalability:** Nodes can be added or removed dynamically.
 - Cloud-native applications often deal with massive volumes of data.
 - Distributed storage enables horizontal scaling by adding more nodes.
 - Example: Azure Data Lake Storage Gen2 scales with petabytes of data.
- **Elasticity:** Resources scale with demand, a key trait of cloud-native systems. The capacity to implement automated scale.
 - Se referă la adaptarea dinamică a resurselor (CPU, memorie, stocare) în funcție de cerințele aplicației.
 - În Azure Databricks, elasticitatea este realizată prin autoscaling-ul clusterelor Spark.
 - Exemplu: un job de procesare batch poate începe cu 2 noduri și scala automat la 20 în funcție de volumul de date.

Data Lake, Delta Lake și LakeHouse – comparații

	Data Lake	Delta Lake	Lakehouse
Definition	Scalable repository for raw, structured and unstructured data.	ACID-compliant storage layer over Data Lake using Parquet and transaction logs	Unified architecture combining Data Lake with Data Warehouse capabilities
File Format	CSV, JSON, Parquet, Avro, etc.	Parquet + _delta_log	Delta Lake, Iceberg+Flink, Hudi, etc.
ACID Transactions	Not natively supported	Fully supported	Integrated via Delta Lake or similar formats
Schema Enforcement	Optional	Strict validation	Complete and automatic
Schema Evolution	Manual	Automatic	Advanced
Time Travel	Not available	Historical versions accessible	Included via Delta Lake/Iceberg
Query Performance	Moderate, depends on format and optimization	High, thanks to indexing and compaction	High, optimized for BI and ML
Streaming + Batch Support	Separate workflows	Unified	Native to architecture
Scalability	Very high	Very high	Very high
Storage Cost	Low	Low	Low
Operational Complexity	Simple but limited	Moderate, requires log management	More complex, but unified and efficient
Ideal Use Cases	Archiving, raw data ingestion	Machine learning, incremental processing, BI	BI, AI, ML, advanced analytics, platform unification

Avro, Iceberg and Hudi a very short comparation

	Apache Avro	Apache Iceberg	Apache Hudi
Type	Compact binary file format row-based serialization with incorporated schema.	Table format for data lakes	Table format for data lakes
Primary Use Case	Data serialization and storage	Large-scale analytics with ACID and time travel	Fast ingestion, incremental updates, and CDC
Storage Format	Row-oriented binary format	Columnar (typically Parquet)	Columnar (typically Parquet)
Schema Evolution	Supported	Supported	Supported
ACID Transactions	Not supported	Fully supported	Fully supported
Time Travel	Not available	Supported	Supported
Partitioning	Manual	Hidden and automatic	Customizable
Streaming Support	Not designed for streaming	Supported	Optimized for streaming and batch
Integration	Kafka, Hadoop, Spark	Spark, Apache Flink(motor de procesare distribuit în timp real)	Spark, Flink, Hive
Indexing	Not available	Limited	Built-in indexing for fast updates
Ideal Scenarios	Compact data exchange and storage	Lakehouse analytics, BI, time travel	Real-time ingestion, CDC(Change Data Capture)

Azure Data Lake Storage Gen2 – Definition

Azure Data Lake Storage Gen2 (ADLS Gen2) is a cloud-native, distributed storage solution designed for big data analytics.

It combines the scalability and durability of **Azure Blob Storage** with the **hierarchical namespace** and performance features of **HDFS**.

It is the primary storage layer for Azure Databricks, supporting structured, semi-structured, and unstructured data, and enabling efficient integration with Spark and Delta Lake.

Abstracția FSSpec

Parquet-formatul în care ADB persista datele în ADL Gen 2

- A DataFrame (df) in Apache Spark is a distributed collection of data organized into columns, similar to an SQL table or a Pandas DataFrame. It is one of the most commonly used data structures in Spark for data processing.
- When write a Parquet file using Spark:
`df.write.parquet ("/mnt/mydata"))`
- the file is stored in the **mounted cloud storage location**, such as ADLS Gen 2
- Also we can write directly to a path like (where **abfss**(azure blob file system secure) `abfss://container@account.dfs.core.windows.net/path` which targets Azure Data Lake Gen2 explicitly

Storage Architecture in Data Bricks

- Storage Architecture in Azure Databricks File System (DBFS): This is an abstraction layer over cloud object storage. In Azure, DBFS is backed by Azure Data Lake Storage Gen2 or Azure Blob Storage, depending on your workspace configuration.
- Spark Cluster: The cluster processes data but does not persist files long-term. Temporary files may exist on the driver or worker nodes during computation, but they are not durable storage locations.

The Persistence in Azure Databricks

Layer	Role in Persistence
ADLS Gen2	Primary external storage for persistent data (Parquet, Delta, etc.)
DBFS (Databricks File System)	Abstraction layer over cloud storage (often backed by ADLS Gen2)
Spark Cluster	Temporary compute layer — does not persist data long-term

Azure DataBricks Storage with Delta Lake

Azure Databricks Storage with Delta Lake is the integration of Delta Lake, a transactional storage layer, with Azure Data Lake Storage Gen2 (ADLS Gen2), used as the underlying distributed file system. This combination enables scalable, reliable, and queryable data storage for big data and machine learning workloads.

- Built on top of Parquet files stored in ADLS Gen2
- Adds ACID (Atomicity, Consistency, Isolation and Durability) transactions, schema enforcement, and time-travel (access to the data history)
- Enables SQL-like querying and version control

Equation: Fault Tolerance = Redundancy + Consensus

1. Redundancy

In **Delta Lake**, redundancy operates on **two levels**:

- **a. Physical data redundancy**

- The Parquet files underlying Delta Lake are stored in **distributed cloud storage** (e.g., AWS S3, Azure Data Lake, GCS), which provides **automatic replication and high availability**.
- This ensures **data durability**.
- In practice, the system can tolerate node or file copy loss without compromising dataset integrity.

- **b. Logical redundancy (multiple versions)**

- Delta Lake maintains a **transaction log** (`_delta_log/`) containing **successive metadata versions**.
- Each commit appends a new JSON (or Parquet checkpoint) file, preserving the **entire history of changes**.
- This is a form of **temporal redundancy** — you can always revert to a consistent previous snapshot (time travel).

This logical redundancy means that processing failures never corrupt the global state, since a valid previous version is always recoverable.

Consensus(1)

This is the subtle but crucial part of Delta Lake's **fault tolerance** mechanism:

a. **Atomicity via implicit consensus protocol**

- Delta Lake enforces a **serialized commit protocol** for `_delta_log`.
- All writes are handled as atomic transactions: only one process can successfully commit at a time.
- The consensus mechanism is implemented via **Optimistic Concurrency Control (OCC)** — each writer validates the current log version before committing.
- If two processes attempt simultaneous writes, only one succeeds; the other must retry.

This acts as a **lightweight distributed consensus**, conceptually similar to **Paxos/Raft**, but applied at the file/version level rather than the cluster node level.

Consensus(2)

b. Consensus on consistent state

- Every reader sees a **snapshot-isolated, consistent view** of the table, based on the latest committed version.
- Thus, all nodes in the cluster (Spark executors, SQL endpoints, etc.) **agree on the same logical truth** of the dataset, even though the physical files may reside on different nodes.

This is the required “consensus” ensuring all distributed actors operate on a consistent truth of data.